

Instituto Tecnológico de Costa Rica  
Centro Académico de Alajuela  
IC6600: Principios de Sistemas Operativos



Tarea 3 - Kick off

Estudiante:

Fabiana Arias Rosales - 2021028380

Ian Calvo Madrigal - 2022136902

Profesor:

MSc. Ing. Kevin Moraga García

Abril 2025

# 1. Introducción

Actualmente, los servidores web se pueden sobrecargar por tener muchas solicitudes de manera simultánea. Existen técnicas para enfrentar este problema y distribuir mejor los recursos como pre-threaded y pre-forked, estos ayudan a mejorar la eficiencia del servidor cuando hay muchos clientes.

Esta tarea consiste en desarrollar un **WebServer** en lenguaje C para GNU/Linux, que implemente el protocolo HTTP/1.1 y utilice la técnica pre-threaded. Esta técnica lo que hace es crear hilos antes de que lleguen las solicitudes, ya que estos van a estar disponibles para responder a los clientes de inmediato. Para esto se usará la biblioteca *pthread*.

El servidor deberá implementar los métodos principales de HTTP:

- GET: Se usa cuando un cliente quiere pedir un archivo o recurso al servidor.
- HEAD: Solicita únicamente los encabezados de un recurso, sin el cuerpo de la respuesta.
- POST: Sirve para enviar información desde el cliente al servidor.
- PUT: Este método permite subir un archivo o reemplazar uno que ya exista en el servidor.
- DELETE: Se utiliza para eliminar un recurso que está almacenado en el servidor.

También, el WebServer deberá tener la capacidad de responder a solicitudes utilizando otros protocolos además de HTTP, como: FTP, SSH, SMTP, DNS, TELNET y SNMP.

Además, se debe desarrollar un **cliente HTTP** en C para realizar pruebas y descargar archivos desde el servidor. También se construirá una herramienta de estrés en Python, para simular muchos clientes al mismo tiempo para saturar el servidor y ver su comportamiento.

## Estrategia para la solución del problema

- Investigar sobre el funcionamiento del protocolo HTTP y los protocolos alternativos a implementar.
- Crear el servidor con hilos previamente inicializados (pre-threaded).
- Desarrollar un cliente para realizar pruebas de comunicación.
- Implementar una herramienta para simular múltiples clientes y evaluar el rendimiento.

## 2. Ambiente de desarrollo

- **Sistema Operativo:** Ubuntu 24.04
- **Lenguaje principal:** C
- **Lenguaje para stress:** Python 3
- **IDE:** Visual Studio Code
- **Debugging:** gdb
- **Control de versiones:** Github

## 3. Control de versiones

Se va a utilizar Github para gestionar el historial del código. El proyecto se almacena en un repositorio en GitHub con el nombre **Tarea3S0**. El enlace al repositorio es el siguiente: <https://github.com/Fabiana0744/Tarea3S0.git>

## 4. Diagrama UML

En este proyecto decidí estructurar en los tres principales componentes: ServidorHTTP, ClienteHTTP y StressCMD.

- **ServidorHTTP:** Este levanta el servidor, esperar conexiones de los clientes y responder a solicitudes utilizando los hilos creados antes.
- **ClienteHTTP:** Este representa al cliente que hace solicitudes al servidor HTTP.
- **StressCMD:** Esta se va a encargar de ejecutar muchas instancias del ClienteHTTP, para estresar al servidor y ver su comportamiento.

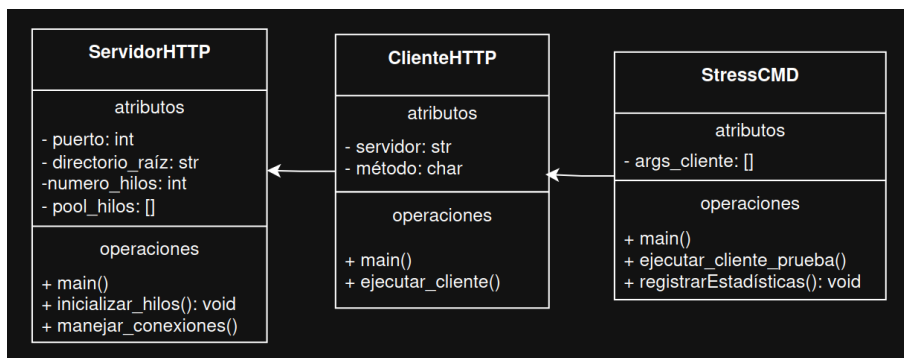


Figura 1: Diagrama de clases UML del sistema propuesto